

BP_EvolutionManager – Descripción limpia del flujo

1. Función general del Blueprint

El **BP_EvolutionManager** es el blueprint encargado de controlar todo el proceso evolutivo de los vehículos inteligentes del programa. Su función principal es gestionar cuándo empieza una simulación, qué coches se generan, con qué red neuronal se inicializa cada uno, cómo se evalúa su rendimiento, qué modelo se guarda como mejor resultado y cuándo se pasa de entrenamiento a test.

Este blueprint actúa como coordinador general del sistema. No conduce directamente los coches, sino que organiza las generaciones, crea los vehículos en los puntos de spawn, les asigna cerebros ya cargados, mutados o aleatorios, espera a que todos terminen o mueran, calcula estadísticas de la generación y decide cuál es el mejor modelo neuronal.

Además, permite trabajar en distintos modos:

- **Modo entrenamiento**, donde se generan coches con cerebros mutados o aleatorios para buscar mejores comportamientos.
- **Modo test**, donde se carga un modelo ya aprendido y se evalúa sin mutarlo.
- **Modo train/test 70/30**, donde una parte de las generaciones se usa para entrenar y la parte final para probar el mejor modelo obtenido.
- **Modo test múltiple**, donde se prueba el modelo cargado en varios spawn points.
- **Modo de coche único**, donde se genera un solo vehículo con el modelo guardado.

2. Variables principales

Variables de configuración de la simulación

`PopulationSize` indica cuántos coches se generan en cada generación. En esta versión se ajusta automáticamente al número de spawn points disponibles después de refrescar la lista de puntos válidos.

`GenerationNumber` almacena la generación actual. Empieza en 1 y aumenta cada vez que termina una generación de entrenamiento o una ronda de test.

`IsSimulating` indica si actualmente hay una simulación activa. Cuando está en `true`, el Event Tick comprueba si todavía quedan coches vivos.

`IsTrainingMode` diferencia si el sistema está entrenando o testeando. Si está en `true`, al morir todos los coches se llama a `EndGeneration`; si está en `false`, se llama a `EndTesting`.

`MultiTestingMode` permite probar el modelo cargado con varios coches y varios puntos de spawn. Si está desactivado, se genera un único vehículo en `LoadBrainSpawnPoint`.

`TrainTest7030Mode` activa el modo mixto de entrenamiento y test. En este modo, el sistema usa el 70% de las generaciones para entrenar y el 30% restante para testear.

`TotalGenerations7030` define el número total de generaciones del modo 70/30.

`TrainGenerationsTarget` guarda cuántas generaciones pertenecen a la fase de entrenamiento. Se calcula como el 70% de `TotalGenerations7030`.

`IsInTrainPhase` indica si el modo 70/30 todavía está en la fase de entrenamiento o si ya ha pasado a la fase de test.

Variables relacionadas con los coches y los spawn points

`CarsArray` almacena todos los vehículos generados durante la generación o test actual.

`SpawnPointsList` contiene los puntos de spawn válidos que se pueden usar en el modo actual.

`SpawnBag` es una bolsa de índices que indica qué spawn point se asigna a cada coche. Después de llenarse, se baraja para repartir los coches de forma aleatoria entre los puntos disponibles.

`LoadBrainSpawnPoint` se usa en el modo de coche único para indicar dónde se generará el vehículo que carga directamente el modelo guardado.

`AiVehicle` y `AiController` son referencias temporales al coche y al controlador que se están creando o configurando en cada momento.

Variables relacionadas con el modelo neuronal

`BestWeights_InputHidden` y `BestWeights_HiddenOutput` almacenan los pesos del mejor modelo encontrado durante la sesión actual.

`Loaded_W_IH` y `Loaded_W_HO` almacenan los pesos cargados desde el modelo guardado previamente en disco.

`BestFitness` guarda el mejor fitness obtenido durante la sesión actual.

`LoadedBestFitness` representa el fitness asociado al mejor modelo cargado desde el slot principal. Se usa para saber si el nuevo mejor modelo debe sustituir al modelo guardado.

`BestCar` guarda la referencia al mejor coche encontrado en la sesión.

`BestCarForWeights` guarda el coche del que se tomarán los pesos para la siguiente generación. Este coche se selecciona teniendo en cuenta la normalización por spawn.

`BestFitnessSpawn` almacena, para cada spawn point, el mejor fitness obtenido en ese spawn. Se utiliza para comparar coches de forma más justa, ya que algunos puntos de inicio pueden ser más difíciles que otros.

`MutationRate` indica la intensidad de mutación aplicada a los pesos de la red neuronal. En `StartGeneration` se fija en `0.07`.

`StartWithModelSaved` determina si la primera generación debe partir de un modelo ya guardado o si debe empezar con cerebros aleatorios.

Variables de resumen y evaluación

`MeanFitness` almacena el fitness medio de la generación o test.

`MeanTime` almacena el tiempo medio de vida de los coches.

`TestMinTime` define el tiempo mínimo que un coche debe sobrevivir para considerarse éxito en test.

`TestMinFitness` define el fitness mínimo requerido en test cuando `TestUseFitness` está activado.

`TestUseFitness` indica si el criterio de éxito en test debe incluir también el fitness mínimo.

3. Flujo inicial: Event BeginPlay

Cuando empieza la simulación, el blueprint crea primero el archivo `Fitness_Debug.csv` con una cabecera que contiene todos los campos de depuración que se van a guardar durante la ejecución. Esta cabecera incluye datos como generación, identificador del coche, fitness final, tiempo vivo, spawn usado, razón de muerte, penalizaciones acumuladas, métricas de parada, valores de mutación y otros datos de análisis.

Después, el sistema comprueba si existe un savegame llamado `SuperCarModel`. Este slot representa el mejor modelo global guardado hasta el momento.

Si el archivo existe, se carga el savegame y se convierte a `BP_NNS`. Si la conversión es válida, se copian sus pesos a `Loaded_W_IH` y `Loaded_W_HO`. También se inicializa `LoadedBestFitness` a `0.0`.

Si el archivo no existe, el sistema crea un nuevo objeto de guardado `BP_NNS`. En este caso, los pesos iniciales del savegame se rellenan con `BestWeights_InputHidden` y `BestWeights_HiddenOutput`, y el fitness inicial se guarda usando `BestFitness`. También se copian esos valores a las variables `Loaded_W_IH`, `Loaded_W_HO` y `LoadedBestFitness`.

Una vez resuelta la carga o creación del modelo, se llama a `RefreshSpawnPointsList`. Esta función actualiza la lista de spawn points válidos según el modo actual.

Después se comprueba si está activado `TrainTest7030Mode`. Si está activado, el sistema fuerza el modo entrenamiento poniendo `IsTrainingMode` en `true`, marca `IsInTrainPhase` como `true` y calcula `TrainGenerationsTarget` como el 70% de `TotalGenerations7030`.

A continuación se crea la cabecera de `Test_Summary.csv`, que se usará para guardar resúmenes de las generaciones de test.

Después el flujo se divide según `IsTrainingMode`:

- Si el sistema está en entrenamiento, se crea la cabecera de `Training_Summary.csv`, se limpia `BestFitnessSpawn`, se rellena con un valor inicial muy bajo para cada spawn point y se llama a `StartGeneration`.
- Si el sistema no está en entrenamiento y `MultiTestingMode` está activado, se crea la cabecera de test y se llama a `StartTesting`.
- Si el sistema no está en entrenamiento y `MultiTestingMode` está desactivado, se genera un único vehículo en `LoadBrainSpawnPoint`, se obtiene su controlador, se pone su `TrainingMode` en `false` y se carga directamente el cerebro guardado mediante `LoadBrain`.

4. Control de simulación: Event Tick

El `Event Tick` se encarga de comprobar continuamente si la simulación actual ha terminado.

Primero pone `CarsAlive` en `false`. Después comprueba si `IsSimulating` está activo. Si no hay simulación activa, no hace nada.

Si hay simulación activa, comprueba que `CarsArray` tenga coches. Si el array está vacío, tampoco continúa.

Cuando hay coches, el comportamiento depende del modo actual:

En modo entrenamiento

El sistema recorre `CarsArray` buscando algún coche que no esté muerto. Para ello comprueba la variable `IsDead` de cada vehículo.

Si encuentra al menos un coche vivo, pone `CarsAlive` en `true` y corta el bucle, porque ya sabe que la generación sigue activa.

Cuando termina la comprobación, si `CarsAlive` sigue en `false`, significa que todos los coches de la generación han terminado. En ese caso:

1. Pone `IsSimulating` en `false`.
2. Incrementa `GenerationNumber`.
3. Espera `0.2` segundos.
4. Llama a `EndGeneration`.

En modo test

El proceso es igual, pero al detectar que todos los coches han terminado se llama a `EndTesting` en lugar de `EndGeneration`.

5. Detención del programa: StopSimulation

El evento `StopSimulation` se usa para finalizar la ejecución completa.

Primero espera `0.2` segundos y después llama a `QuitGame`, usando el `PlayerController(0)`. El cierre se realiza con preferencia `Quit` y permitiendo ignorar restricciones de plataforma.

Este evento se usa especialmente al finalizar todas las generaciones del modo 70/30.

6. Inicio de una generación: StartGeneration

`StartGeneration` prepara y lanza una nueva generación de entrenamiento.

Primero fija `MutationRate` en `0.07` y limpia `CarsArray`, eliminando las referencias a coches anteriores.

Si `GenerationNumber` es mayor que 1, se vuelve a llamar a `RefreshSpawnPointsList`. Esto permite actualizar los spawn points disponibles antes de crear la nueva población.

Después se calcula cómo se repartirá la población:

- `NumMutatedEach = Truncate((PopulationSize - 2) * 0.4)`
- `MutLoadedEnd = NumMutatedEach + 2`
- `MutSessionEnd = MutLoadedEnd + NumMutatedEach`

La idea es reservar los dos primeros coches para modelos sin mutación y usar después dos bloques de coches mutados: uno a partir del modelo cargado y otro a partir del mejor modelo de la sesión. El resto de la población se inicializa con cerebros aleatorios.

Antes de generar coches, se comprueba que existan spawn points y que `SpawnBag` tenga elementos. Si ambas listas son válidas, se ejecuta un bucle desde `1` hasta `PopulationSize`.

En cada iteración se genera un `BP_VehicleAI2`. Su posición se obtiene a partir del spawn point correspondiente en `SpawnBag`. Además, durante entrenamiento se añade una pequeña variación lateral usando el vector derecho del spawn y un valor aleatorio entre `-25` y `25`. También se añade una variación aleatoria en la rotación Z entre `-5` y `5` grados. Esto hace que los coches no empiecen siempre exactamente en la misma posición y orientación, aumentando la variabilidad del entrenamiento.

Después de crear el coche:

1. Se le asigna un controlador con `SpawnDefaultController`.
2. Se guarda la referencia del vehículo en `AiVehicle`.
3. Se asigna su `SpawnPoint`.
4. Se asigna su `GenerationCount` con el número de generación actual.
5. Se obtiene su controlador y se convierte a `BP_AiController`.
6. Se guarda el índice del coche en `Debug_CarIndex`.

A continuación se decide cómo inicializar el cerebro del coche.

Primera generación

Si `GenerationNumber` no es mayor que 1, el sistema está en la primera generación.

En este caso se comprueba `StartWithModelSaved`:

- Si `StartWithModelSaved` es `false`, el coche se inicializa con `InitializeRandomBrain`. Esto crea un cerebro aleatorio.
- Si `StartWithModelSaved` es `true`, el coche se marca como `SimulationWithLoadedCar`, se le aplica el color de coche cargado mediante `LoadedCarColor` y se inicializa con `InitializeMutatedBrain` usando los pesos cargados `Loaded_W_IH` y `Loaded_W_HO`, pero con `MutationRate = 0.0`. Es decir, se usa el modelo guardado sin modificarlo.

En ambos casos se guardan valores de depuración relacionados con la mutación: número de pesos cambiados, magnitud media de cambio, número de cambios grandes y fitness del padre.

Generaciones posteriores

Si `GenerationNumber` es mayor que 1, el sistema reparte la población en varios grupos.

El coche con `LoopIndex == 1` usa el modelo cargado desde `SuperCarModel1` sin mutación. Se le aplica `LoadedCarColor` y se inicializa con `Loaded_W_IH` y `Loaded_W_HO` con `MutationRate = 0.0`. Su fitness de padre es `LoadedBestFitness`.

El coche con `LoopIndex == 2` usa el mejor modelo encontrado durante la sesión actual, también sin mutación. Se le aplica `BestCarColor` y se inicializa con `BestWeights_InputHidden` y `BestWeights_HiddenOutput` con `MutationRate = 0.0`. Su fitness de padre es `BestFitness`.

Los coches entre `LoopIndex >= 3` y `LoopIndex <= MutLoadedEnd` se generan a partir del modelo cargado, pero aplicando mutación con `MutationRate`. Estos coches exploran variaciones del mejor modelo global guardado.

Los coches entre `LoopIndex >= MutLoadedEnd + 1` y `LoopIndex <= MutSessionEnd` se generan a partir del mejor modelo de la sesión actual, también aplicando mutación con `MutationRate`. Estos coches exploran variaciones del mejor individuo encontrado durante la ejecución actual.

El resto de coches se inicializan con `InitializeRandomBrain`, por lo que introducen diversidad aleatoria en la población.

Finalmente, cada coche generado se añade a `CarsArray` y se activa `IsSimulating` en `true`.

7. Final de una generación: EndGeneration

`EndGeneration` se ejecuta cuando todos los coches de una generación de entrenamiento han muerto o han terminado.

Primero llama a `SortByFitness`, que ordena `CarsArray` de mayor a menor fitness.

Después llama a `NormalizeFitness`. Esta función selecciona `BestCarForWeights`, que será el coche del que se tomarán los pesos para la evolución. La selección no depende únicamente del fitness absoluto, sino de una comparación normalizada por spawn.

Luego llama a `UpdateFitnessPerSpawn`, que actualiza el mejor fitness histórico obtenido en cada spawn point.

Después se guarda `BestCarForWeights` como `BestCarRef` y se toma su fitness como `CurrentBestFitness`.

A continuación se llama a `CalculateAndExport`, que calcula estadísticas de la generación y las escribe en el archivo correspondiente.

Después se comprueba si `CurrentBestFitness` supera a `BestFitness`.

Si no mejora el mejor fitness de la sesión

Si el nuevo coche no supera el mejor fitness de la sesión, no se actualizan pesos ni savegames. El sistema pasa directamente a destruir los coches y decidir si debe empezar otra generación o pasar a test.

Si mejora el mejor fitness de la sesión

Si `CurrentBestFitness > BestFitness`, el sistema actualiza el mejor coche de la sesión:

1. `BestCar` pasa a ser `BestCarRef`.
2. `BestFitness` pasa a ser `CurrentBestFitness`.
3. Se obtiene el `BP_AiController` del mejor coche.
4. Se copian sus pesos a `BestWeights_InputHidden` y `BestWeights_HiddenOutput`.
5. Se crea un savegame `BP_NNS`.
6. Se guardan los pesos y el fitness en ese objeto.
7. Se guarda el modelo en el slot `SessionCarModel`.

Después se comprueba si `BestFitness` también supera a `LoadedBestFitness`.

Si lo supera, significa que el nuevo modelo es mejor que el mejor modelo global cargado al inicio. En ese caso:

1. Se crea otro objeto `BP_NNS`.
2. Se guardan los pesos y el fitness actual.
3. Se guarda una copia de seguridad con nombre `SuperCarModel_prev_Día-Mes_Hora-Minuto`.
4. Se sobrescribe el slot principal `SuperCarModel`.
5. Se actualizan `LoadedBestFitness`, `Loaded_W_IH` y `Loaded_W_HO` con los nuevos valores.

De esta forma, el sistema mantiene tanto el mejor modelo de la sesión como el mejor modelo global.

Limpieza y decisión siguiente

Después de evaluar y guardar si es necesario, el sistema recorre `CarsArray` y destruye todos los vehículos.

Cuando termina de destruirlos, comprueba si está activo el modo 70/30, si todavía estaba en fase de entrenamiento y si `GenerationNumber` ya ha superado `TrainGenerationsTarget`.

- Si la condición es falsa, se llama de nuevo a `StartGeneration` y continúa el entrenamiento.
- Si la condición es verdadera, se cambia a modo test: `IsTrainingMode` pasa a `false` e `IsInTrainPhase` pasa a `false`. Luego se intenta cargar `SessionCarModel`, se copian sus pesos a `Loaded_W_IH` y `Loaded_W_HO`, y finalmente se llama a `StartTesting`.

8. Ordenación por fitness: SortByFitness

`SortByFitness` ordena el array de coches de mayor a menor fitness.

Para ello usa dos bucles anidados. En cada comparación, si el coche de la posición actual tiene menos fitness que el coche siguiente, se intercambian sus posiciones mediante `Swap`.

El resultado es que, al terminar la función, `CarsArray[0]` contiene el coche con mayor fitness bruto de la generación.

9. Normalización del fitness: `NormalizeFitness`

`NormalizeFitness` intenta seleccionar el mejor coche de forma más justa teniendo en cuenta el spawn point en el que ha empezado.

Primero inicializa `BestNormFitness` con un valor muy bajo y asigna provisionalmente `BestCarForWeights` al primer coche de `CarsArray`.

Después recorre todos los coches. Para cada coche:

1. Busca en `SpawnPointsList` el índice del spawn point usado por ese vehículo.
2. Comprueba que ese índice sea válido dentro de `BestFitnessSpawn`.
3. Calcula un valor normalizado usando `SafeDivide(Fitness del coche, BestFitnessSpawn del spawn)`.
4. Si ese valor normalizado es mayor que `BestNormFitness`, actualiza `BestNormFitness` y asigna ese coche como `BestCarForWeights`.

La finalidad de esta normalización es evitar que el sistema elija siempre coches de spawn points más fáciles. En lugar de comparar únicamente el fitness absoluto, compara el rendimiento del coche frente al mejor rendimiento conocido en su propio punto de salida.

10. Actualización del mejor fitness por spawn: `UpdateFitnessPerSpawn`

`UpdateFitnessPerSpawn` actualiza el array `BestFitnessSpawn`.

La función recorre todos los coches de `CarsArray`. Para cada uno, busca qué índice tiene su `SpawnPoint` dentro de `SpawnPointsList`.

Si el índice es válido, se actualiza la posición correspondiente de `BestFitnessSpawn` con el máximo entre:

- El fitness actual del coche.
- El mejor fitness ya almacenado para ese spawn.

Así, cada spawn point conserva el mejor resultado histórico conseguido desde ese punto. Estos valores se utilizan después en `NormalizeFitness` para calcular una comparación relativa por spawn.

11. Cálculo y exportación de estadísticas: CalculateAndExport

`CalculateAndExport` calcula el resumen de una generación o test y lo guarda en el archivo CSV correspondiente.

Primero inicializa varias variables locales:

- `MaxNormFitness`, para guardar el fitness del coche seleccionado como mejor.
- `SumaFitness`, para acumular el fitness total.
- `SumaTiempo`, para acumular el tiempo vivo total.
- `ContadorMuertes`, para contar cuántas veces aparece cada razón de muerte.
- `PopularDeath`, para guardar la razón de muerte más repetida.
- `DeathCount`, para guardar cuántas veces aparece esa razón.
- `BestCarDeathReason`, para guardar la razón de muerte del mejor coche.
- `SuccessCount` y `FailCount`, para contar éxitos y fracasos en test.
- `SuccessRate` y `FailRate`, para calcular porcentajes.

Después recorre todos los coches de `CarsArray`.

Para cada coche, guarda su referencia en `LocalCar`, lee su `DeathReason` y su `Fitness`. Luego actualiza el mapa `ContadorMuertes`:

- Si esa razón de muerte ya existía en el mapa, incrementa su contador.
- Si no existía, la añade con valor `1`.

Después suma el tiempo vivo del coche a `SumaTiempo` y su fitness a `SumaFitness`.

A continuación decide si ese coche cuenta como éxito o fracaso. Se considera fracaso si cumple alguna de estas condiciones:

- Su razón de muerte es `LAZY`.
- Su razón de muerte es `REVERSE`.
- Su razón de muerte es `CORRECTINGWRONG`.
- Su razón de muerte es `REVERSING_WRONG`.
- Su razón de muerte es `TRAFFIC_VIOLATION`.
- Su razón de muerte es `STOP_VIOLATION`.
- Su razón de muerte es `YIELD_VIOLATION`.
- Su razón de muerte es `INVALID_BRAIN`.
- Su razón de muerte empieza por `COLLISION_WITH_`.
- Está activado `TestUseFitness` y el fitness del coche es menor que `TestMinFitness`.
- El tiempo vivo del coche es menor que `TestMinTime`.

Si alguna de esas condiciones se cumple, aumenta `FailCount`. Si no se cumple ninguna, aumenta `SuccessCount`.

También se comprueba si el coche actual es `BestCarForWeights`. Si lo es, se guarda su fitness en `MaxNormFitness` y su razón de muerte en `BestCarDeathReason`.

Cuando termina el bucle de coches, el sistema recorre las claves de `ContadorMuertes` para encontrar la razón de muerte más frecuente. La razón con mayor contador se guarda en `PopularDeath` y su número de apariciones se guarda en `DeathCount`.

Después se calculan los promedios y porcentajes:

- `MeanFitness = SumaFitness / número de coches`
- `MeanTime = SumaTiempo / número de coches`
- `SuccessRate = SuccessCount * 100 / número de coches`
- `FailRate = FailCount * 100 / número de coches`

Finalmente, el sistema guarda una línea en el CSV correspondiente.

Si está en entrenamiento, escribe en `Training_Summary.csv`:

- Generación.
- Mejor fitness.
- Fitness medio.
- Tiempo medio.
- Razón de muerte del mejor coche.
- Razón de muerte más frecuente.
- Número de muertes de la razón más frecuente.
- Número total de coches.

Si está en test, escribe en `Test_Summary.csv` esos mismos datos y además:

- Número de éxitos.
- Número de fracasos.
- Porcentaje de éxito.
- Porcentaje de fallo.

12. Inicio del test: StartTesting

`StartTesting` prepara una ronda de evaluación usando el modelo cargado, sin aplicar mutaciones.

Primero limpia `CarsArray`. Si `GenerationNumber` es mayor que 1, vuelve a llamar a `RefreshSpawnPointsList` para actualizar los spawn points disponibles.

Después comprueba que existan spawn points y que `SpawnBag` tenga elementos. Si la comprobación es correcta, ejecuta un bucle desde `1` hasta `PopulationSize`.

En cada iteración genera un `BP_VehicleAI2` en el transform exacto del spawn point correspondiente. A diferencia del entrenamiento, aquí no se añade variación lateral ni rotación aleatoria, porque el objetivo del test es evaluar el modelo de forma controlada.

Después de generar el coche:

1. Se le asigna un controlador con `SpawnDefaultController`.
2. Se guarda la referencia en `AiVehicle`.
3. Se le asigna su `SpawnPoint`.
4. Se le asigna `GenerationCount`.
5. Se obtiene su controlador y se convierte a `BP_AiController`.
6. Se pone `TrainingMode` en `false`.
7. Se aplica `LoadedCarColor`.
8. Se llama a `LoadBrain`, cargando `Loaded_W_IH` y `Loaded_W_H0`.
9. Se añade el coche a `CarsArray`.

Al final se pone `IsSimulating` en `true`, dejando que el `Event Tick` controle cuándo termina el test.

13. Final del test: EndTesting

`EndTesting` se ejecuta cuando todos los coches de una ronda de test han terminado.

Primero llama a `SortByFitness`, ordenando los coches por fitness bruto.

Después selecciona `CarsArray[0]` como mejor coche del test, asignándolo a `BestCarRef` y a `BestCarForWeights`. También guarda su fitness en `CurrentBestFitness`.

Luego llama a `CalculateAndExport`, que calcula las estadísticas del test y las guarda en `Test_Summary.csv`.

Después destruye todos los coches de `CarsArray`.

Cuando termina la limpieza, comprueba si está activo el modo 70/30, si ya no está en fase de entrenamiento y si `GenerationNumber` ha superado `TotalGenerations7030`.

- Si la condición es verdadera, llama a `StopSimulation` y finaliza el programa.
 - Si la condición es falsa, llama de nuevo a `StartTesting` para iniciar otra ronda de test.
-

14. Actualización de spawn points: RefreshSpawnPointsList

`RefreshSpawnPointsList` reconstruye la lista de puntos de spawn disponibles.

Primero limpia `SpawnPointsList` y `SpawnBag`.

Después obtiene todos los actores de clase `BP_SpawnPoints` que existen en el nivel. Para cada uno, comprueba si la referencia es válida.

Si el sistema está en modo entrenamiento, solo añade a `SpawnPointsList` los spawn points que tengan `ValidForTrain` activado.

Si el sistema no está en modo entrenamiento, solo añade los spawn points que tengan `ValidForTest` activado.

Cuando termina de revisar todos los spawn points, ajusta `PopulationSize` al número de spawn points válidos encontrados.

Si `PopulationSize` es mayor que cero, rellena `SpawnBag` con índices desde `0` hasta `PopulationSize - 1`. Cada índice representa una posición dentro de `SpawnPointsList`.

Finalmente, baraja `SpawnBag` usando `ShuffleArray`. De esta forma, los coches no se asignan siempre a los spawn points en el mismo orden.

15. Resumen del flujo completo

El funcionamiento general del **BP_EvolutionManager** puede entenderse como un ciclo.

Primero, al iniciar el juego, el sistema prepara los archivos de salida, carga el mejor modelo guardado si existe y recopila los spawn points válidos. Después decide si debe empezar entrenando, testeando o generando un único coche con el modelo cargado.

Durante entrenamiento, `StartGeneration` crea una población de vehículos. Algunos usan el modelo guardado, otros usan el mejor modelo de la sesión, otros son mutaciones de esos modelos y otros se inicializan de forma aleatoria. Cada coche conduce de forma independiente hasta morir o terminar.

Mientras la simulación está activa, el `Event Tick` comprueba si queda algún coche vivo. Cuando todos han terminado, llama a `EndGeneration`.

En `EndGeneration`, los coches se ordenan por fitness, se selecciona el mejor candidato teniendo en cuenta la normalización por spawn, se actualizan estadísticas, se guardan modelos si hay mejora y se destruyen los vehículos. Después el sistema decide si iniciar otra generación o pasar a test.

Durante test, `StartTesting` genera coches usando siempre el modelo cargado, sin mutaciones. Al terminar, `EndTesting` calcula estadísticas de éxito, fracaso, fitness y tiempo medio. Si todavía quedan generaciones de test, inicia otra ronda; si ya se ha completado el modo 70/30, detiene el programa.

En conjunto, este blueprint controla la evolución completa del comportamiento de los coches: carga modelos previos, genera variaciones, evalúa resultados, conserva los mejores pesos y permite comprobar el rendimiento final en modo test.